

Space Telemetry Ground Station - Revue technique du code

Document de presentation pour recruteur informatique

Project: `cpp-space-telemetry-ground-station`

Date: 3 July 2026

Ce document explique le projet comme je souhaiterais qu'un recruteur technique le lise: non pas comme un simple exercice C++, mais comme une demonstration de ma maniere de concevoir un logiciel robuste. Les extraits de code presentes ci-dessous sont volontairement choisis pour montrer la logique de conception, les choix algorithmiques, la complexite, la gestion des erreurs, le decoupage applicatif et la philosophie generale de developpement.

Table des matières

1	Resume executif pour recruteur	2
1.1	Ce que le projet cherche a prouver	2
1.2	Cartographie rapide des competences	2
2	Philosophie generale de code	2
3	Architecture globale	2
3.1	Decoupage logique	2
3.2	Flux de donnees	3
4	Conception du protocole binaire	3
4.1	Extrait: structure d'une trame	3
4.2	Extrait: ecriture big-endian portable	3
5	Encodage et validation des trames	4
5.1	Extrait: encodage defensif	4
5.2	Extrait: decodage strict et erreurs explicites	5
6	Controle d'integrite CRC32	6
6.1	Extrait: table CRC32 compile-time	6
7	Extraction de trames depuis un flux TCP	6
7.1	Extrait: reconstitution depuis un stream	6
8	Multithreading: pipeline producteur/consommateur	7
8.1	Extrait: queue bloquante generique	7
8.2	Extrait: orchestration des workers	8
9	Mode degraded: surveillance operationnelle	8
9.1	Extrait: fenetre glissante et hysteresis	8
10	Replay binaire STGF	9
10.1	Extrait: format de replay defensif	9
11	Export CSV/JSON sans dependance externe	10
11.1	Extrait: echappement JSON et ecriture de sortie	10
12	Reception reseau Linux	11
12.1	Extrait: reception UDP avec poll	11
13	Benchmark et analyse de performance	12
13.1	Extrait: benchmark de pipeline	12
13.2	Mesures obtenues dans l'environnement de generation	12
14	Tests et qualite	13
14.1	Extrait: tests unitaires sur protocole et erreurs	13
14.2	Extrait: CMake et CTest	13
15	Limites connues et evolutions possibles	14
16	Ce que le document doit faire comprendre d'un point de vue recruteur	14
17	Conclusion	14

1 Resume executif pour recruteur

Space Telemetry Ground Station est une mini station sol de telemetry spatiale developpee en C++20 sous Linux. Le projet recoit des trames binaires simulees en UDP/TCP ou depuis un fichier de replay, les decode, verifie leur integrite par CRC32, les exporte en CSV/JSON et surveille l'etat operationnel de la station avec un mode **DEGRADED**.

L'objectif n'etait pas de faire une interface graphique, mais de construire une base logicielle credible pour un environnement industriel: protocole binaire strict, separation des couches, CMake propre, tests, replay, benchmark, logs, multithreading et documentation.

1.1 Ce que le projet cherche a prouver

- capacite a ecrire du C++ moderne, lisible et testable;
- comprehension des contraintes Linux, reseau, fichiers binaires et robustesse;
- capacite a concevoir un protocole clair et verifiable;
- capacite a raisonner en performance: complexite, debits, effets de contention;
- approche industrielle: erreurs explicites, logs, replay, benchmark, CI et documentation.

1.2 Cartographie rapide des competences

Competence	Ou elle apparait dans le projet
C++20 moderne	<code>std::span</code> , <code>std::variant</code> , RAII, types forts, separation bibliotheque/applications
Linux	sockets POSIX, <code>poll()</code> , signaux, fichiers binaires, build CMake
Reseau	reception UDP, flux TCP, extraction de trames depuis un stream
Robustesse	CRC32, magic bytes, controle de version, longueurs strictes, erreurs detaillees
Multithreading	queues bloquantes, workers de decodage, thread d'ecriture
Performance	benchmark reproductible, analyse de debit et de contention
Tests	tests unitaires C++ et integration CTest
Documentation	README, design technique, rapport performance, ce document

2 Philosophie generale de code

Ma philosophie dans ce projet repose sur cinq idees.

1. **Rendre les limites explicites.** Les tailles, versions, longueurs, formats et seuils sont visibles dans le code, pas caches dans des constantes implicites.
2. **Isoler les responsabilites.** Le codec ne fait pas de reseau; le reseau ne connait pas la semantique metier; l'application orchestre les threads et les exports.
3. **Fail fast, mais sans crash inutile.** Une trame invalide est rejetee avec un code d'erreur precis; le service peut continuer a traiter les trames suivantes.
4. **Preferer la dependance minimale.** Le projet n'utilise pas de grosse bibliotheque externe pour le JSON ou le reseau afin de montrer la comprehension des mecanismes bas niveau.
5. **Mesurer au lieu d'affirmer.** Le benchmark et le rapport de performance montrent que le comportement est observable.

3 Architecture globale

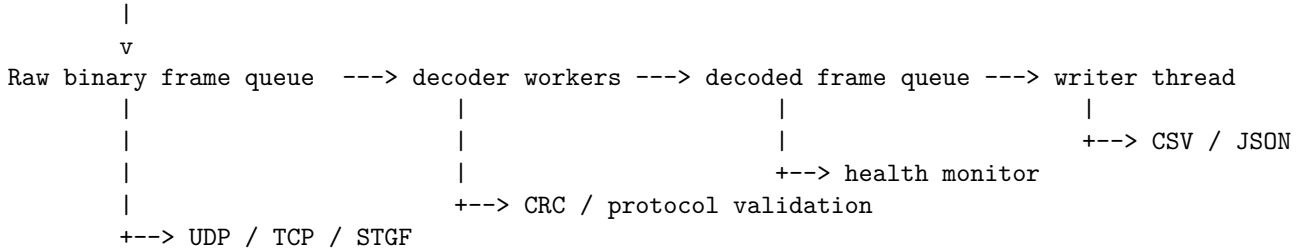
3.1 Decoupage logique

Composant	Role
<code>stgs_core</code>	Protocole, CRC32, codec, replay, logs, monitoring de sante
<code>stgs_network</code>	Reception UDP/TCP Linux via sockets POSIX
<code>stgs_ground_station</code>	Application principale: orchestration, threads, export CSV/JSON
<code>stgs_satellite_simulator</code>	Simulateur de trames, pertes, corruption, generation STGF

stgs_benchmark_decode	Mesure reproductible du debit de decodage
stgs_tests	Tests unitaires du coeur fonctionnel

3.2 Flux de donnees

Satellite simulator / Replay file



Ce flux a ete volontairement pense comme une chaine de traitement industrielle: chaque etape a une responsabilite claire, ce qui rend le code plus facile a tester, a maintenir et a faire evoluer.

4 Conception du protocole binaire

4.1 Extrait: structure d'une trame

```

1 namespace stgs {
2
3 using ByteVector = std::vector<std::uint8_t>;
4
5 inline constexpr std::uint32_t FrameMagic = 0x53544753U; // ASCII "STGS"
6 inline constexpr std::uint8_t ProtocolVersion = 1U;
7 inline constexpr std::size_t MagicSize = 4U;
8 inline constexpr std::size_t HeaderSize = 23U;
9 inline constexpr std::size_t CrcSize = 4U;
10 inline constexpr std::size_t MinFrameSize = HeaderSize + CrcSize;
11 inline constexpr std::size_t MaxPayloadSize = 4096U;
12 inline constexpr std::size_t MaxFrameSize = HeaderSize + MaxPayloadSize + CrcSize;
13
14 // Binary frame layout, all integer fields are big-endian:
15 // MAGIC:u32 | VERSION:u8 | SATELLITE_ID:u16 | TIMESTAMP_MS:u64 |
16 // TEMPERATURE_C:f32 | BATTERY_PERCENT:u8 | STATUS:u8 | PAYLOAD_LEN:u16 |
17 // PAYLOAD:bytes | CRC32:u32
18
19 enum class Status : std::uint8_t {
20     Nominal = 0,
21     Warning = 1,
22     Critical = 2,
23     SafeMode = 3
24 };
25
26 struct TelemetryFrame {
27     std::uint8_t version = ProtocolVersion;
28     std::uint16_t satellitelid = 0;
29     std::uint64_t timestampMs = 0;

```

Listing 1: Structure de trame binaire et constantes de protocole

La trame contient un magic fixe, une version, un identifiant satellite, un timestamp, des donnees metier, une longueur de payload et un CRC final. Ce choix permet au decodeur de repondre a trois questions avant d'accepter une trame: *est-ce le bon protocole ?*, *est-ce la bonne version ?*, *le contenu est-il complet et integre ?*

Complexity: La lecture d'un en-tete est en $O(1)$ car les champs sont a offsets fixes. La lecture du payload est en $O(p)$ ou p est la taille du payload. La memoire de la trame decodee est en $O(p)$.

Rationale: Le format binaire est plus strict qu'un JSON reseau: il force des tailles, des bornes et une interpretation unique. C'est coherent avec une logique telemetry/industrielle ou chaque octet a une signification.

4.2 Extrait: ecriture big-endian portable

```

1 inline void appendU8(std::vector<std::uint8_t>& out, std::uint8_t value) {
2     out.push_back(value);
3 }
4
5 inline void appendU16BE(std::vector<std::uint8_t>& out, std::uint16_t value) {
6     out.push_back(static_cast<std::uint8_t>((value >> 8U) & 0xFFU));
7     out.push_back(static_cast<std::uint8_t>(value & 0xFFU));
8 }
9
10 inline void appendU32BE(std::vector<std::uint8_t>& out, std::uint32_t value) {
11     out.push_back(static_cast<std::uint8_t>((value >> 24U) & 0xFFU));

```

```

12     out.push_back(static_cast<std::uint8_t>((value >> 16U) & 0xFFU));
13     out.push_back(static_cast<std::uint8_t>((value >> 8U) & 0xFFU));
14     out.push_back(static_cast<std::uint8_t>(value & 0xFFU));
15 }
16
17 inline void appendU64BE(std::vector<std::uint8_t>& out, std::uint64_t value) {
18     for (int shift = 56; shift >= 0; shift -= 8) {
19         out.push_back(static_cast<std::uint8_t>((value >> static_cast<unsigned>(shift)) & 0xFFU));
20     }
21 }
22
23 inline void appendFloatBE(std::vector<std::uint8_t>& out, float value) {
24     const auto bits = std::bit_cast<std::uint32_t>(value);
25     appendU32BE(out, bits);
26 }
27
28 inline std::uint16_t readU16BE(std::span<const std::uint8_t> bytes, std::size_t offset) {
29     return static_cast<std::uint16_t>((static_cast<std::uint16_t>(bytes[offset]) << 8U) |
30                                     static_cast<std::uint16_t>(bytes[offset + 1U]));
31 }

```

Listing 2: Serialisation big-endian explicite

J'ai choisi le big-endian, souvent appele *network byte order*, pour eviter que le format depende de l'architecture machine. Le code ne fait pas de `reinterpret_cast` direct d'une structure C++ vers des octets, car cela exposerait le protocole aux problemes d'alignement, de padding et d'endianness.

Complexity: Chaque champ numerique est encode en $O(k)$ ou k est le nombre d'octets du champ. Comme k vaut 1, 2, 4 ou 8, c'est $O(1)$ par champ. Pour une trame complete, l'encodage est domine par la copie du payload: $O(p)$.

Rationale: Ce choix est plus verbeux qu'un cast memoire, mais beaucoup plus robuste et portable. C'est un choix de qualite logicielle plutot que de raccourci.

5 Encodage et validation des trames

5.1 Extrait: encodage defensif

```

1     if (i > 0U) {
2         oss << ' ';
3     }
4     oss << std::setw(2) << static_cast<int>(payload[i]);
5 }
6 if (payload.size() > displayed) {
7     oss << " ...(" << std::dec << payload.size() << " bytes)";
8 }
9 return oss.str();
10 }
11
12 std::string toString(const TelemetryFrame& frame) {
13     std::ostringstream oss;
14     oss << "sat=" << frame.satelliteId
15         << " ts_ms=" << frame.timestampMs
16         << " temp_c=" << std::fixed << std::setprecision(2) << frame.temperatureC
17         << " battery=" << static_cast<int>(frame.batteryPercent) << "%"
18         << " status=" << statusToString(frame.status)
19         << " payload_len=" << frame.payload.size();
20     return oss.str();
21 }
22
23 ByteVector encodeFrame(const TelemetryFrame& frame) {
24     if (frame.payload.size() > MaxPayloadSize) {
25         throw std::invalid_argument("payload exceeds MaxPayloadSize");
26     }
27     if (frame.version != ProtocolVersion) {
28         throw std::invalid_argument("unsupported protocol version");
29     }
30     if (frame.batteryPercent > 100U) {
31         throw std::invalid_argument("battery percent must be between 0 and 100");
32     }
33
34     const auto statusByte = static_cast<std::uint8_t>(frame.status);
35     if (statusByte > static_cast<std::uint8_t>(Status::SafeMode)) {

```

Listing 3: Encodage d une trame avec validations prealables

L'encodage commence par valider les invariants: taille maximale du payload, version supportee, batterie dans la plage 0..100, statut connu. L'objectif est de ne jamais produire volontairement une trame invalide.

Complexity: L'encodage reserve la memoire une fois, ajoute des champs fixes en $O(1)$, copie le payload en $O(p)$, puis calcule le CRC sur la trame en $O(h + p)$, avec h la taille fixe de l'en-tete. Donc la complexite totale est $O(p)$.

Rationale: La validation est placee a l'entree de la fonction de codec. Cela centralise les regles du protocole: si demain une autre application genere des trames, elle reutilise les memes garde-fous.

5.2 Extrait: decodage strict et erreurs explicites

```

1  }
2
3  ByteVector bytes;
4  bytes.reserve(HeaderSize + frame.payload.size() + CrcSize);
5  detail::appendU32BE(bytes, FrameMagic);
6  detail::appendU8(bytes, frame.version);
7  detail::appendU16BE(bytes, frame.satelliteId);
8  detail::appendU64BE(bytes, frame.timestampMs);
9  detail::appendFloatBE(bytes, frame.temperatureC);
10 detail::appendU8(bytes, frame.batteryPercent);
11 detail::appendU8(bytes, statusByte);
12 detail::appendU16BE(bytes, static_cast<std::uint16_t>(frame.payload.size()));
13 bytes.insert(bytes.end(), frame.payload.begin(), frame.payload.end());
14
15 const auto crc = crc32(bytes);
16 detail::appendU32BE(bytes, crc);
17 return bytes;
18 }
19
20 FrameParseResult decodeFrame(std::span<const std::uint8_t> bytes) {
21     if (bytes.size() < MinFrameSize) {
22         return makeError(FrameErrorCode::TooShort, "frame is shorter than the minimum header + CRC size");
23     }
24
25     const auto magic = detail::readU32BE(bytes, 0U);
26     if (magic != FrameMagic) {
27         return makeError(FrameErrorCode::BadMagic, "invalid frame magic; expected ASCII STGS");
28     }
29
30     const auto version = bytes[4U];
31     if (version != ProtocolVersion) {
32         return makeError(FrameErrorCode::UnsupportedVersion, "unsupported protocol version");
33     }
34
35     const auto payloadLength = detail::readU16BE(bytes, 21U);
36     if (payloadLength > MaxPayloadSize) {
37         return makeError(FrameErrorCode::PayloadTooLarge, "payload length exceeds configured maximum");
38     }
39
40     const auto expectedSize = HeaderSize + static_cast<std::size_t>(payloadLength) + CrcSize;
41     if (bytes.size() != expectedSize) {
42         std::ostringstream oss;
43         oss << "frame length mismatch: expected " << expectedSize << " bytes, got " << bytes.size();
44         return makeError(FrameErrorCode::LengthMismatch, oss.str());
45     }
46
47     const auto expectedCrc = detail::readU32BE(bytes, bytes.size() - CrcSize);
48     const auto calculatedCrc = crc32(bytes.subspan(0U, bytes.size() - CrcSize));
49     if (expectedCrc != calculatedCrc) {
50         std::ostringstream oss;
51         oss << "CRC mismatch: expected 0x" << std::hex << std::setw(8) << std::setfill('0') << expectedCrc
52             << ", calculated 0x" << std::setw(8) << calculatedCrc;
53         return makeError(FrameErrorCode::BadCrc, oss.str());
54     }
55
56     const auto battery = bytes[19U];
57     if (battery > 100U) {
58         return makeError(FrameErrorCode::InvalidBattery, "battery percent is outside the 0..100 range");
59     }
60
61     const auto statusByte = bytes[20U];
62     if (statusByte > static_cast<std::uint8_t>(Status::SafeMode)) {
63         return makeError(FrameErrorCode::InvalidStatus, "status field contains an unknown value");
64     }
65 }

```

Listing 4: Decodage strict et production d erreurs metier

Le decodeur est ecrit comme une succession de verrous de securite. Il rejette rapidement les trames trop courtes, avec mauvais magic, version non supportee, payload trop grand, taille incoherente, CRC invalide ou champs metier invalides.

Un point important: le resultat est un `std::variant<TelemetryFrame, FrameError>`. Cela evite de melanger les exceptions et le flux normal de traitement. Une trame invalide n'est pas forcement une erreur fatale du programme: c'est un evenement attendu dans un systeme reseau.

Complexity: Le decodage est en $O(p)$ car le CRC doit parcourir tous les octets hors CRC. Les controles d'en-tete sont en $O(1)$. La copie du payload est en $O(p)$.

Trade-off: L'utilisation d'un `variant` rend l'appelant responsable de traiter les deux cas. C'est plus explicite qu'un retour nul ou qu'une exception lancee pour chaque trame corrompue.

6 Controle d'integrite CRC32

6.1 Extrait: table CRC32 compile-time

```

1      std::array<std::uint32_t, 256> table{};
2      for (std::uint32_t i = 0; i < table.size(); ++i) {
3          std::uint32_t crc = i;
4          for (int bit = 0; bit < 8; ++bit) {
5              if ((crc & 1U) != 0U) {
6                  crc = (crc >> 1U) ^ 0xEDB88320U;
7              } else {
8                  crc >>= 1U;
9              }
10         }
11         table[i] = crc;
12     }
13     return table;
14 }
15
16 constexpr auto CrcTable = makeTable();
17
18 } // namespace
19
20 std::uint32_t crc32(std::span<const std::uint8_t> bytes) noexcept {
21     std::uint32_t crc = 0xFFFFFFFFU;
22     for (const auto byte : bytes) {
23         const auto index = static_cast<std::uint8_t>((crc ^ byte) & 0xFFU);
24         crc = (crc >> 8U) ^ CrcTable[index];
25     }

```

Listing 5: CRC32 table-driven

Le CRC32 utilise une table de 256 entrees generee en `constexpr`. La table evite de recalculer les huit etapes bit a bit pour chaque octet a l'execution. Le polynome `0xEDB88320` correspond a une variante classique de CRC32.

Complexity: Le calcul est en $O(n)$ avec n le nombre d'octets. La memoire supplementaire est $O(1)$: une table fixe de 256 entiers 32 bits, soit environ 1 KiB. Sans table, le calcul resterait $O(n)$ mais avec un facteur constant plus eleve.

Rationale: Le CRC32 n'est pas un mecanisme cryptographique. Il sert ici a detecter corruption, troncature ou modification accidentelle d'une trame. Pour une authentication de messages, il faudrait ajouter une signature ou un MAC.

7 Extraction de trames depuis un flux TCP

7.1 Extrait: reconstitution depuis un stream

```

1      return "LengthMismatch";
2      case FrameErrorCode::BadCrc:
3          return "BadCrc";
4      case FrameErrorCode::InvalidBattery:
5          return "InvalidBattery";
6      case FrameErrorCode::InvalidStatus:
7          return "InvalidStatus";
8      }
9      return "Unknown";
10 }
11
12 std::vector<ByteVector> StreamFrameExtractor::feed(std::span<const std::uint8_t> bytes) {
13     buffer_.insert(buffer_.end(), bytes.begin(), bytes.end());
14     std::vector<ByteVector> frames;
15     const auto magic = magicBytes();
16
17     while (true) {
18         const auto it = std::search(buffer_.begin(), buffer_.end(), magic.begin(), magic.end());
19         if (it == buffer_.end()) {
20             if (buffer_.size() > MagicSize - 1U) {
21                 buffer_.erase(buffer_.begin(), buffer_.end() - static_cast<std::ptrdiff_t>(MagicSize - 1U));
22             }
23             return frames;
24         }
25
26         if (it != buffer_.begin()) {
27             buffer_.erase(buffer_.begin(), it);
28         }
29
30         if (buffer_.size() < HeaderSize) {
31             return frames;
32         }
33
34         const auto payloadLength = detail::readU16BE(buffer_, 21U);
35         if (payloadLength > MaxPayloadSize) {
36             buffer_.erase(buffer_.begin());
37             continue;

```

```

38     }
39
40     const auto frameSize = HeaderSize + static_cast<std::size_t>(payloadLength) + CrcSize;
41     if (buffer_.size() < frameSize) {
42         return frames;
43     }
44
45     frames.emplace_back(buffer_.begin(), buffer_.begin() + static_cast<std::ptrdiff_t>(frameSize));
46     buffer_.erase(buffer_.begin(), buffer_.begin() + static_cast<std::ptrdiff_t>(frameSize));
47 }

```

Listing 6: Extraction de trames completes depuis un flux TCP

UDP conserve une notion de datagramme: une reception correspond a un paquet candidat. TCP, lui, fournit un flux d'octets sans frontiere de message. Le `StreamFrameExtractor` garde donc un buffer interne, cherche le magic STGS, attend d'avoir l'en-tete complet, lit la longueur du payload, puis extrait uniquement les trames completes.

Complexity: Pour chaque feed, l'insertion est en $O(n)$ avec n octets recus. La recherche du magic via `std::search` est en pratique proche de $O(n)$ car le motif fait 4 octets. Le pire cas theorique est $O(n \cdot m)$ avec $m = 4$, donc assimilable a $O(n)$.

Trade-off: Cette implementation privilegie la lisibilite. Pour un tres haut debit, on pourrait remplacer le buffer vector + erase par un ring buffer afin de reduire les copies et les deplacements memoire.

8 Multithreading: pipeline producteur/consommateur

8.1 Extrait: queue bloquante generique

```

1  class BlockingQueue {
2  public:
3      BlockingQueue() = default;
4      BlockingQueue(const BlockingQueue&) = delete;
5      BlockingQueue& operator=(const BlockingQueue&) = delete;
6
7      bool push(T value) {
8          {
9              std::lock_guard<std::mutex> lock(mutex_);
10             if (closed_) {
11                 return false;
12             }
13             queue_.push(std::move(value));
14         }
15         cv_.notify_one();
16         return true;
17     }
18
19     std::optional<T> pop() {
20         std::unique_lock<std::mutex> lock(mutex_);
21         cv_.wait(lock, [this] { return closed_ || !queue_.empty(); });
22         if (queue_.empty()) {
23             return std::nullopt;
24         }
25         T value = std::move(queue_.front());
26         queue_.pop();
27         return value;
28     }
29
30     void close() {
31         {
32             std::lock_guard<std::mutex> lock(mutex_);
33             closed_ = true;
34         }
35         cv_.notify_all();
36     }
37
38     [[nodiscard]] bool closed() const {
39         std::lock_guard<std::mutex> lock(mutex_);
40         return closed_;
41     }
42
43     [[nodiscard]] std::size_t size() const {
44         std::lock_guard<std::mutex> lock(mutex_);
45         return queue_.size();
46     }
47
48 private:
49     mutable std::mutex mutex_;

```

Listing 7: Queue thread-safe avec fermeture explicite

La queue bloquante permet de decoupler la reception, le decodage et l'ecriture. Le choix de `std::condition_variable` evite le busy-wait: les threads dorment tant qu'il n'y a rien a traiter.

Complexity: `push()` et `pop()` sont en $O(1)$ amorti. La memoire est $O(q)$ avec q le nombre d'elements en attente. La latence depend surtout de la contention sur le mutex et de la taille des objets copies/deplaces.

Rationale: La methode `close()` est essentielle: elle permet aux threads consommateurs de sortir proprement sans signal artificiel dans la queue. C'est un choix de robustesse et de lisibilite.

8.2 Extrait: orchestration des workers

```

1      std::thread writerThread([&] {
2          bool firstJsonFrame = true;
3          while (auto frame = decodedQueue.pop()) {
4              if (outputFormat == OutputFormat::Json) {
5                  writeFrameJson(out, *frame, firstJsonFrame);
6                  firstJsonFrame = false;
7              } else {
8                  writeFrameCsv(out, *frame);
9              }
10             ++written;
11         }
12         if (outputFormat == OutputFormat::Json) {
13             writeJsonFooter(out);
14         }
15         out.flush();
16     });

17     std::vector<std::thread> decoderThreads;
18     decoderThreads.reserve(opts.decoderThreads);
19     for (std::size_t i = 0; i < opts.decoderThreads; ++i) {
20         decoderThreads.emplace_back([&, i] {
21             while (auto bytes = rawQueue.pop()) {
22                 auto result = stgs::decodeFrame(*bytes);
23                 if (std::holds_alternative<stgs::TelemetryFrame>(result)) {
24                     auto frame = std::get<stgs::TelemetryFrame>(std::move(result));
25                     ++decoded;
26                     if (auto transition = health.recordDecoded(frame); transition.has_value()) {
27                         logTransition(logger, *transition);
28                     }
29                     decodedQueue.push(std::move(frame));
30                 } else {
31                     const auto& err = std::get<stgs::FrameError>(result);
32                     ++rejected;
33                     if (auto transition = health.recordRejected(); transition.has_value()) {
34                         logTransition(logger, *transition);
35                     }
36                     logger.warning("decoder " + std::to_string(i) + " rejected frame: " +
37                                   stgs::errorCodeToString(err.code) + " - " + err.message);
38                 }
39             }
40         });
41     }
42 }

```

Listing 8: Workers de decodage et thread d ecriture

Le pipeline separe deux couts: le cout CPU du decodage/CRC et le cout I/O de l'ecriture CSV/JSON. Les workers consomment des trames binaires, appellent `decodeFrame()`, puis poussent les trames valides vers une autre queue.

Complexity: Pour N trames de payload moyen p , le travail total est $O(N \cdot p)$. Avec T workers, le temps ideal tend vers $O((N \cdot p)/T)$, mais uniquement tant que la contention sur la queue, les allocations et l'I/O ne deviennent pas dominantes.

Trade-off: Le benchmark montre qu'ajouter trop de threads peut degrader les performances: dans ce projet, 4 threads sont moins efficaces que 1 ou 2 sur le test donne. C'est un bon exemple de raisonnement industriel: le multithreading doit etre mesure, pas suppose benefique.

9 Mode degraded: surveillance operationnelle

9.1 Extrait: fenetre glissante et hysteresis

```

1      throw std::invalid_argument("recovery rejection rate must be between 0 and 1");
2  }
3  if (config_.recoveryRejectionRate > config_.degradedRejectionRate) {
4      throw std::invalid_argument("recovery rejection rate cannot exceed degraded rejection rate");
5  }
6  }

7
8  std::optional<StationStateTransition> StationHealthMonitor::recordDecoded(const TelemetryFrame& frame) {
9      return recordSample(Sample{false, isCriticalTelemetry(frame)});
10 }
11
12 std::optional<StationStateTransition> StationHealthMonitor::recordRejected() {
13     return recordSample(Sample{true, false});
14 }
15
16 std::optional<StationStateTransition> StationHealthMonitor::recordSample(Sample sample) {
17     std::lock_guard<std::mutex> lock(mutex_);
18     if (!config_.enabled) {
19         return std::nullopt;

```

```

20     }
21
22     samples_.push_back(sample);
23     while (samples_.size() > config_.windowSize) {
24         samples_.pop_front();
25     }
26
27     const auto snapshot = makeSnapshotLocked();
28     if (snapshot.samples < config_.minSamples) {
29         return std::nullopt;
30     }
31
32     StationState target = state_;
33     if (state_ == StationState::Nominal) {
34         if (snapshot.rejectionRate >= config_.degradedRejectionRate ||
35             snapshot.critical >= config_.criticalFramesForDegraded) {
36             target = StationState::Degraded;
37         }
38     } else {
39         if (snapshot.rejectionRate <= config_.recoveryRejectionRate && snapshot.critical == 0U) {
40             target = StationState::Nominal;
41         }
42     }
43
44     if (target == state_) {
45         return std::nullopt;
46     }
47
48     StationStateTransition transition;
49     transition.from = state_;
50     transition.to = target;
51     transition.snapshot = snapshot;
52     transition.reason = transitionReasonLocked(snapshot, target);
53     state_ = target;
54     transition.snapshot.state = state_;
55     return transition;
56 }

```

Listing 9: Changement d etat NOMINAL / DEGRADED

Le mode DEGRADED represente l'etat de la station, pas seulement le statut d'une trame. Il est declenche si le taux de rejet depasse un seuil ou si plusieurs trames critiques apparaissent dans la fenetre de surveillance. Le retour a NOMINAL utilise un seuil plus bas: c'est une hysteresis, utile pour eviter les oscillations rapides.

Complexity: La version actuelle parcourt la fenetre pour calculer un snapshot: $O(w)$ par echantillon, avec w la taille de fenetre. Par default $w = 100$, donc le cout est borne et tres faible. Une optimisation future consisterait a maintenir des compteurs incrementaux pour passer en $O(1)$ par echantillon.

Rationale: J'ai privilegie une implementation facile a relire et auditer. Dans un contexte critique, un algorithme legerement moins optimise mais comprehensible peut etre preferable, tant que le cout est borne et mesure.

10 Replay binaire STGF

10.1 Extrait: format de replay defensif

```

1 void FrameFileWriter::writeFrame(std::span<const std::uint8_t> frameBytes) {
2     if (frameBytes.size() > MaxFrameSize) {
3         throw std::runtime_error("refusing to write frame larger than MaxFrameSize");
4     }
5     ByteVector length;
6     detail::appendU32BE(length, static_cast<std::uint32_t>(frameBytes.size()));
7     out_.write(reinterpret_cast<const char*>(length.data()), static_cast<std::streamsize>(length.size()));
8     out_.write(reinterpret_cast<const char*>(frameBytes.data()), static_cast<std::streamsize>(frameBytes.size()));
9     if (!out_) {
10         throw std::runtime_error("failed while writing replay frame");
11     }
12 }
13
14 FrameFileReader::FrameFileReader(const std::filesystem::path& path) {
15     in_.open(path, std::ios::binary | std::ios::in);
16     if (!in_) {
17         throw std::runtime_error("failed to open replay input file: " + path.string());
18     }
19
20     std::array<std::uint8_t, FileHeader.size()> header{};
21     readExact(in_, header.data(), header.size());
22     if (header != FileHeader) {
23         throw std::runtime_error("invalid or unsupported STGF replay file header");
24     }
25 }
26
27 std::optional<ByteVector> FrameFileReader::readNext() {
28     std::array<std::uint8_t, 4> lenBytes{};
29     in_.read(reinterpret_cast<char*>(lenBytes.data()), static_cast<std::streamsize>(lenBytes.size()));
30     const auto bytesRead = in_.gcount();
31     if (bytesRead == 0 && in_.eof()) {
32         return std::nullopt;

```

```

33     }
34     if (bytesRead != static_cast<std::streamsize>(lenBytes.size())) {
35         throw std::runtime_error("truncated frame length in replay file");
36     }
37
38     const auto length = detail::readU32BE(lenBytes, 0U);
39     if (length < MinFrameSize || length > MaxFrameSize) {
40         throw std::runtime_error("invalid replay frame length");
41     }
42
43     ByteVector frame(length);
44     readExact(in_, frame.data(), frame.size());
45     return frame;

```

Listing 10: Lecture et ecriture de fichiers STGF

Le replay sert a rendre les incidents reproductibles. Un fichier **STGF** commence par un header fixe, puis stocke chaque trame precedee de sa longueur. La lecture verifie le header, la taille minimale, la taille maximale et les fins de fichier tronquees.

Complexity: L'ecriture et la lecture sont en $O(n)$ pour n octets stockes. Chaque trame ajoute un overhead constant de 4 octets de longueur.

Rationale: Le replay est une fonctionnalite importante pour le debug industriel: au lieu de dire “ca a bugge en reseau”, on capture les trames et on rejoue exactement le meme scenario.

11 Export CSV/JSON sans dependance externe

11.1 Extrait: echappement JSON et ecriture de sortie

```

1  std::string jsonEscape(const std::string& value) {
2      std::ostringstream out;
3      for (unsigned char ch : value) {
4          switch (ch) {
5              case '"':
6                  out << "\\\"";
7                  break;
8              case '\\':
9                  out << "\\\"";
10                 break;
11                 case '\b':
12                     out << "\\b";
13                     break;
14                     case '\f':
15                         out << "\\f";
16                         break;
17                         case '\n':
18                             out << "\\n";
19                             break;
20                             case '\r':
21                                 out << "\\r";
22                                 break;
23                                 case '\t':
24                                     out << "\\t";
25                                     break;
26                             default:
27                                 if (ch < 0x20U) {
28                                     out << "\\u" << std::hex << std::setw(4) << std::setfill('0') << static_cast<int>(ch);
29                                 } else {
30                                     out << static_cast<char>(ch);
31                                 }
32                                 break;
33                             }
34                         }
35                     return out.str();
36                 }
37
38                 void writeCsvHeader(std::ofstream& out) {
39                     out << "timestamp_ms,satellite_id,temperature_c,battery_percent,status,payload_len,payload_hex\n";
40                 }
41
42                 void writeFrameCsv(std::ofstream& out, const stgs::TelemetryFrame& frame) {
43                     out << frame.timestampMs << ','
44                         << frame.satelliteId << ','
45                         << frame.temperatureC << ','
46                         << static_cast<int>(frame.batteryPercent) << ','
47                         << stgs::statusToString(frame.status) << ','
48                         << frame.payload.size() << ','
49                         << csvEscape(stgs::payloadToHex(frame.payload, frame.payload.size())) << '\n';
50                 }
51
52                 void writeJsonHeader(std::ofstream& out) {
53                     out << "[\n";
54                 }
55
56                 void writeFrameJson(std::ofstream& out, const stgs::TelemetryFrame& frame, bool first) {
57                     if (!first) {

```

```

58     out << ",\n";
59 }
60 out << " {"
61 << "\"timestamp_ms\":\"" << frame.timestampMs << ','
62 << "\"satellite_id\":\"" << frame.satelliteId << ','
63 << "\"temperature_c\":\"" << std::setprecision(6) << frame.temperatureC << ','
64 << "\"battery_percent\":\"" << static_cast<int>(frame.batteryPercent) << ','
65 << "\"status\":\"" << jsonEscape(stgs::statusToString(frame.status)) << ','
66 << "\"payload_len\":\"" << frame.payload.size() << ','
67 << "\"payload_hex\":\"" << jsonEscape(stgs::payloadToHex(frame.payload, frame.payload.size())) << "\"
68 << '}'
69 }
70
71 void writeJsonFooter(std::ofstream& out) {
72     out << "\n]\n";

```

Listing 11: Export CSV/JSON

L'export JSON a été implémenté sans bibliothèque externe pour garder le projet portable. Le point sensible est l'échappement des chaînes: guillemets, backslashes, caractères de contrôle. Le CSV est également échappé pour éviter de casser les colonnes lorsque le payload est affiché en hex.

Complexity: L'échappement est en $O(s)$ avec s la taille de la chaîne. L'export total est en $O(N \cdot p)$ parce que la conversion du payload en hexadécimal parcourt le payload.

Trade-off: Pour une application de production plus riche, une bibliothèque JSON reconnue serait préférable. Ici le choix manuel est pédagogique: il montre la compréhension du format et limite les dépendances.

12 Reception reseau Linux

12.1 Extrait: reception UDP avec poll

```

1
2     int yes = 1;
3     if (::setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes)) < 0) {
4         ::close(fd);
5         throw std::runtime_error("setsockopt(SO_REUSEADDR) failed: " + std::string(std::strerror(errno)));
6     }
7
8     const auto addr = makeAddress(config_.bindAddress, config_.port);
9     if (::bind(fd, reinterpret_cast<const sockaddr*>(&addr), sizeof(addr)) < 0) {
10         ::close(fd);
11         throw std::runtime_error("bind() failed on " + config_.bindAddress + ":" +
12             std::to_string(config_.port) + ": " + std::strerror(errno));
13     }
14
15     return fd;
16 }
17
18 void NetworkServer::runUdp(FrameCallback& callback, const std::atomic_bool& running) {
19     serverFd_ = createBoundSocket(SOCK_DGRAM);
20     logger_.info("UDP receiver listening on " + config_.bindAddress + ":" + std::to_string(config_.port));
21
22     ByteVector buffer(MaxFrameSize);
23     while (running.load() && !stopRequested_.load()) {
24         pollfd pfd{serverFd_, POLLIN, 0};
25         const int rc = ::poll(&pfd, 1, config_.pollTimeoutMs);
26         if (rc < 0) {
27             if (errno == EINTR) {
28                 continue;
29             }
30             throw std::runtime_error("poll() failed: " + std::string(std::strerror(errno)));
31         }
32         if (rc == 0 || (pfd.revents & POLLIN) == 0) {
33             continue;
34         }
35
36         sockaddr_in src{};
37         socklen_t srcLen = sizeof(src);
38         const ssize_t n = ::recvfrom(serverFd_, buffer.data(), buffer.size(), 0,
39             reinterpret_cast<sockaddr*>(&src), &srcLen);
40         if (n < 0) {

```

Listing 12: Boucle UDP Linux avec poll

La réception UDP utilise des sockets POSIX et `poll()` avec timeout. Cela permet d'attendre des données sans bloquer indéfiniment, tout en laissant la boucle vérifier le signal d'arrêt. Les erreurs transitoires comme `EINTR` sont gérées sans arrêter le serveur.

Complexity: Pour UDP, chaque datagramme est traité en $O(f)$ avec f la taille de la trame copiée. Pour TCP, la boucle de polling est en $O(c)$ par cycle avec c clients connectés.

Trade-off: `poll()` est simple et portable sur Linux/Unix. Pour un serveur avec des milliers de clients, `epoll()` serait plus adapté. Ici, le projet vise une station de telemetry démonstrative, pas un serveur web massif.

13 Benchmark et analyse de performance

13.1 Extrait: benchmark de pipeline

```

1  int main(int argc, char** argv) {
2      try {
3          const auto opts = parseArgs(argc, argv);
4          std::mt19937 rng(opts.seed);
5          std::vector<stgs::ByteVector> encoded;
6          encoded.reserve(opts.frames);
7          for (std::size_t i = 0; i < opts.frames; ++i) {
8              encoded.push_back(stgs::encodeFrame(makeFrame(opts, rng, i)));
9          }
10
11         stgs::BlockingQueue<stgs::ByteVector> queue;
12         std::atomic_size_t decoded{0};
13         std::atomic_size_t rejected{0};
14
15         std::vector<std::thread> workers;
16         workers.reserve(opts.decoderThreads);
17         const auto start = std::chrono::steady_clock::now();
18         for (std::size_t i = 0; i < opts.decoderThreads; ++i) {
19             workers.emplace_back([&] {
20                 while (auto bytes = queue.pop()) {
21                     auto result = stgs::decodeFrame(*bytes);
22                     if (std::holds_alternative<stgs::TelemetryFrame>(result)) {
23                         ++decoded;
24                     } else {
25                         ++rejected;
26                     }
27                 }
28             });
29         }
30
31         for (const auto& frame : encoded) {
32             queue.push(frame);
33         }
34         queue.close();
35         for (auto& worker : workers) {
36             worker.join();
37         }
38         const auto end = std::chrono::steady_clock::now();
39         const auto seconds = std::chrono::duration<double>(end - start).count();
40         const auto fps = static_cast<double>(decoded.load() + rejected.load()) / seconds;
41
42         std::cout << "benchmark=decode_pipeline"
43                   << " frames=" << opts.frames
44                   << " payload_size=" << opts.payloadSize
45                   << " decoder_threads=" << opts.decoderThreads
46                   << " decoded=" << decoded.load()
47                   << " rejected=" << rejected.load()
48                   << " duration_seconds=" << seconds
49                   << " frames_per_second=" << fps << '\n';

```

Listing 13: Benchmark decode pipeline

Le benchmark genere des trames valides, les pousse dans la queue, lance plusieurs workers et mesure le nombre de trames traitees par seconde. Il verifie aussi que toutes les trames attendues sont decodees et qu'aucune n'est rejetee.

Complexity: Le benchmark mesure un travail en $O(N \cdot p)$, avec N trames et p taille payload. La comparaison des debits selon p montre l'impact du CRC et des copies de payload.

13.2 Mesures obtenues dans l'environnement de generation

Frames	Payload	Decoder threads	Measured throughput
200000	64 B	1	1091780 frames/s
200000	64 B	2	1133140 frames/s
200000	64 B	4	135736 frames/s
100000	16 B	2	1035120 frames/s
100000	64 B	2	1238090 frames/s
100000	256 B	2	438356 frames/s
100000	1024 B	2	305913 frames/s
100000	4096 B	2	93070 frames/s

Ces mesures ne sont pas une promesse de performance absolue: elles dependent de la machine, du compilateur et de la charge systeme. Elles sont toutefois utiles pour comparer les tendances.

- Le payload de 4096 B reduit fortement le debit, ce qui est logique car le CRC32 et la copie sont lineaires.
- Le passage de 1 a 2 threads apporte peu mais reste positif sur le cas 64 B.

- Le passage a 4 threads degrade fortement le resultat mesure: contention de queue, scheduling et copies dominant probablement.

La conclusion technique est importante: **le multithreading n'est pas automatiquement une optimisation**. La bonne approche consiste a benchmarker, puis a ajuster l'architecture.

14 Tests et qualite

14.1 Extrait: tests unitaires sur protocole et erreurs

```

1  auto parsed = stgs::decodeFrame(bytes);
2  ASSERT_TRUE(std::holds_alternative<stgs::FrameError>(parsed));
3  ASSERT_EQ(std::get<stgs::FrameError>(parsed).code, stgs::FrameErrorCode::BadCrc);
4  }
5
6  void testLengthMismatch() {
7      auto bytes = stgs::encodeFrame(sampleFrame());
8      bytes.pop_back();
9      auto parsed = stgs::decodeFrame(bytes);
10     ASSERT_TRUE(std::holds_alternative<stgs::FrameError>(parsed));
11     ASSERT_EQ(std::get<stgs::FrameError>(parsed).code, stgs::FrameErrorCode::LengthMismatch);
12 }
13
14 void testStreamExtractorSplitAndNoise() {
15     const auto frameA = stgs::encodeFrame(sampleFrame());
16     auto frameBValue = sampleFrame();
17     frameBValue.satelliteId = 99;
18     frameBValue.payload = {1, 2, 3};
19     const auto frameB = stgs::encodeFrame(frameBValue);
20
21     stgs::StreamFrameExtractor extractor;
22     stgs::ByteVector chunk1 = {0x00, 0x11, 0x22};
23     chunk1.insert(chunk1.end(), frameA.begin(), frameA.begin() + 10);

```

Listing 14: Tests sur CRC, round-trip et erreurs

Les tests couvrent plusieurs proprietes essentielles: vecteur connu CRC32, encode/decode round-trip, mauvais magic, mauvais CRC, taille incoherente, extraction TCP, replay, mode degraded et queue bloquante.

Complexity: Les tests de codec sont en $O(p)$ par trame, comme le code teste. Les tests cherchent surtout a valider des invariants, pas a prouver mathematiquement tout l'espace d'entree.

Rationale: Les tests ne sont pas seulement la pour augmenter un pourcentage de couverture. Ils documentent les garanties attendues du protocole: une trame corrompue doit etre refusee, une trame valide doit etre restituee fidelement.

14.2 Extrait: CMake et CTest

```

1  cmake_minimum_required(VERSION 3.16)
2
3  project(cpp_space_telemetry_ground_station
4      DESCRIPTION "C++20 Linux telemetry ground station simulator"
5      LANGUAGES CXX)
6
7  set(CMAKE_CXX_STANDARD 20)
8  set(CMAKE_CXX_STANDARD_REQUIRED ON)
9  set(CMAKE_CXX_EXTENSIONS OFF)
10
11 option(STGS_BUILD_TESTS "Build unit tests" ON)
12 option(STGS_BUILD_BENCHMARKS "Build benchmark tools" ON)
13
14 find_package(Threads REQUIRED)
15
16 set(STGS_WARNINGS -Wall -Wextra -Wpedantic)
17 if(CMAKE_CXX_COMPILER_ID STREQUAL "GNU")
18     list(APPEND STGS_WARNINGS -Wno-free-nonheap-object)
19 endif()
20
21 add_library(stgs_core
22     src/Crc32.cpp
23     src/FrameCodec.cpp
24     src/Logger.cpp
25     src/Replay.cpp
26     src/StationHealth.cpp)
27
28 target_include_directories(stgs_core
29     PUBLIC
30     ${CMAKE_CURRENT_SOURCE_DIR}/include)
31
32 target_compile_options(stgs_core PRIVATE ${STGS_WARNINGS})
33
34 add_library(stgs_network
35     src/NetworkServer.cpp)
36

```

```

37 target_link_libraries(stgs_network
38     PUBLIC
39     stgs_core
40     Threads::Threads)
41
42 target_compile_options(stgs_network PRIVATE ${STGS_WARNINGS})
43
44 add_executable(stgs_ground_station
45     apps/ground_station.cpp)
46
47 target_link_libraries(stgs_ground_station
48     PRIVATE
49     stgs_network
50     stgs_core
51     Threads::Threads)

```

Listing 15: Structure CMake: libraries, executables, tests, benchmarks

Le projet est separe en bibliotheques et executables. Cela evite de lier toute l'application pour tester le coeur. C'est aussi plus propre pour une evolution future: un autre outil pourrait reutiliser `stgs_core` sans embarquer la couche reseau.

15 Limites connues et evolutions possibles

Limite actuelle	Evolution industrielle possible
Queue basee sur mutex unique	Ring buffer lock-free ou queue MPMC optimisee si le benchmark l'exige
Snapshot degraded en $O(w)$	Compteurs incrementaux pour passer en $O(1)$ par echantillon
JSON manuel	Bibliotheque JSON robuste si le format s'enrichit
<code>poll()</code> pour TCP	<code>epoll()</code> si grand nombre de clients
CRC32 non cryptographique	HMAC/signature si besoin d'authenticite
Pas de persistance base de donnees	Ajout PostgreSQL ou timeseries DB pour historisation
Pas de packaging Debian	Ajout install target, systemd service, paquet Linux

16 Ce que le document doit faire comprendre d'un point de vue recruteur

Le projet montre une logique de developpeur qui ne se limite pas a "faire fonctionner" un programme. Les choix visibles sont ceux d'un logiciel que l'on veut expliquer, tester, rejouer, mesurer et maintenir.

- J'accepte la complexite utile: protocole binaire, CRC, replay, multithreading.
- Je refuse la complexite gratuite: pas de framework lourd pour masquer les mecanismes.
- Je pense en erreurs: corruption, troncature, version invalide, fermeture propre des threads.
- Je pense en exploitation: logs, mode degraded, replay, benchmark.
- Je pense en recrutement technique: le code doit etre lisible par quelqu'un qui ne m'a jamais parle.

17 Conclusion

Cette station sol de telemetry est un projet portfolio oriente C++ industriel. Il ne pretend pas etre un produit spatial certifie, mais il demontre une base concrete: C++20, Linux, reseau, protocole binaire, CRC32, multithreading, tests, benchmark et documentation. Sa valeur principale est de montrer ma philosophie de code: explicite, mesurable, robuste, documentee et orientee maintenance.